Simulate API: JSON → validation → DTOs → Cirq → response

This page shows how a **POST /api/simulate** body travels through the backend. The **level-definition** shape (`GateStep` / `steps[]`) lives *outside* this HTTP flow—it is used for the built-in **target library**, not for the trial circuit JSON.

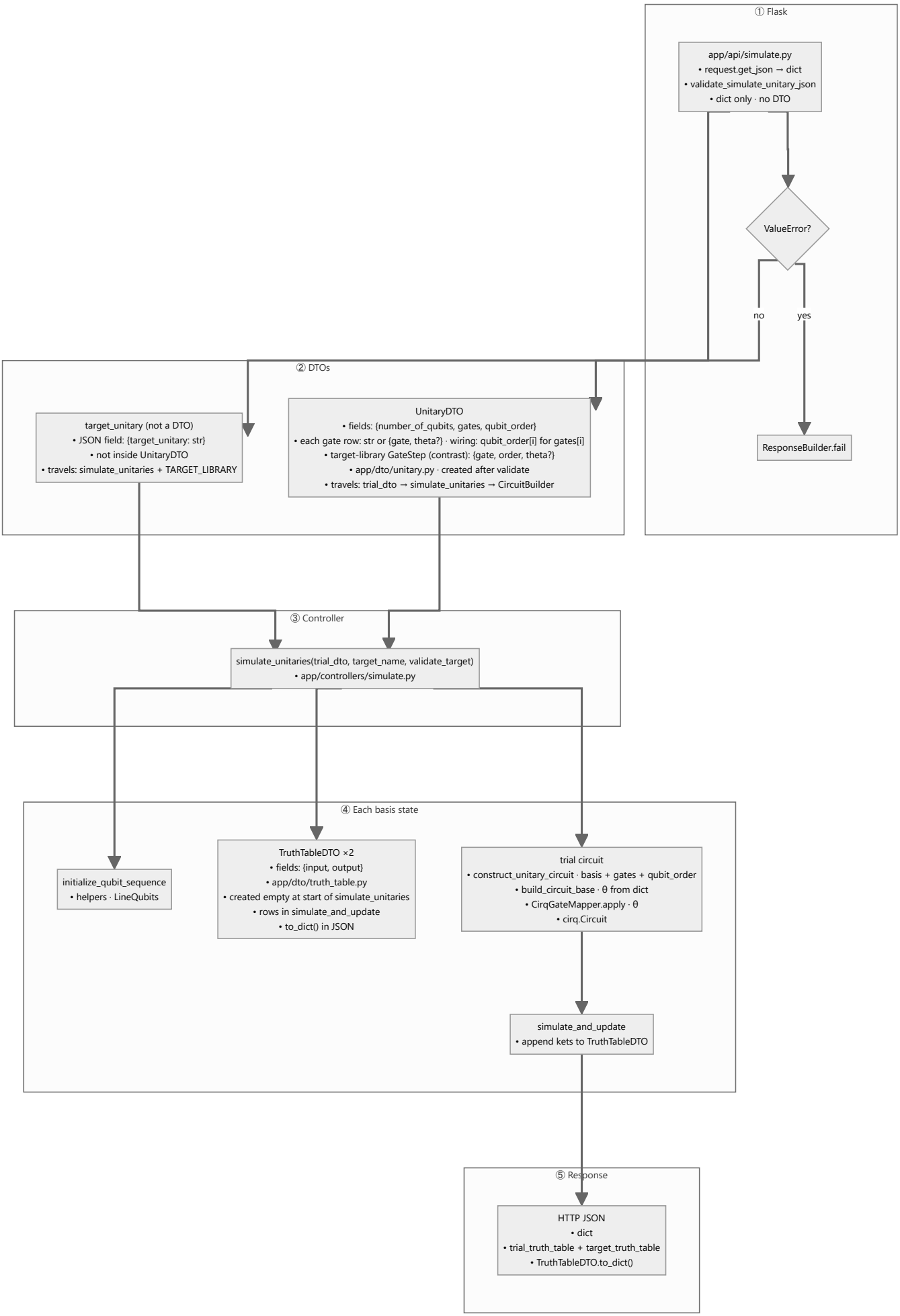
Example request body (includes `theta` on one gate)

```
{
  "number_of_qubits": 2,
  "gates": ["H", { "gate": "RX", "theta": 1.2 }, "CNOT"],
  "qubit_order": [[0], [1], [0, 1]],
  "target_unitary": "SWAP"
}
```

Row `i`: `gates[i]` pairs with `qubit_order[i]` (H on q0, RX(1.2) on q1, CNOT 0→1).

Flowchart + level definition (sidebar)

Main path: trial circuit from HTTP


① Flask
app/api/simulate.py
• request.get_json → dict
• validate_simulate_unitary_json
• dict only · no DTO
ValueError?
no
yes
ResponseBuilder.fail
② DTOs
target_unitary (not a DTO)
• JSON field: {target_unitary: str}
• not inside UnitaryDTO
• travels: simulate_unitaries + TARGET_LIBRARY
UnitaryDTO
• fields: {number_of_qubits, gates, qubit_order}
• each gate row: str or {gate, theta?} · wiring: qubit_order[i] for gates[i]
• target-library GateStep (contrast): {gate, order, theta?}
• app/dto/unitary.py · created after validate
• travels: trial_dto → simulate_unitaries → CircuitBuilder
③ Controller
simulate_unitaries(trial_dto, target_name, validate_target)
• app/controllers/simulate.py
④ Each basis state
initialize_qubit_sequence
• helpers · LineQubits
TruthTableDTO ×2
• fields: {input, output}
• app/dto/truth_table.py
• created empty at start of simulate_unitaries
• rows in simulate_and_update
• to_dict() in JSON
trial circuit
• construct_unitary_circuit · basis + gates + qubit_order
• build_circuit_base · θ from dict
• CirqGateMapper.apply · θ
• cirq.Circuit
simulate_and_update
• append kets to TruthTableDTO
⑤ Response
HTTP JSON
• dict
• trial_truth_table + target_truth_table
• TruthTableDTO.to_dict()

Files: `app/api/simulate.py` → `app/utils/unitary_payload.py` → `app/dto/unitary.py` → `app/controllers/simulate.py` → `app/services/circuit_builder.py` → `app/config/gates.py` → `app/services/simulator.py`.

Level definition — not in this request

Canonical targets are authored as Python data with **self-contained steps**. Each step includes **order** on the same object (no parallel `qubit_order` array).

`app/utils/types.py — GateStep (TypedDict)`

- `gate` (string)
- `order` (list of qubit indices)
- `theta` optional — same meaning as in the HTTP dict gate

`app/config/target_library.py`

- `TARGET_LIBRARY[name]` has `steps`, `num_qubits`, `expected_outputs`, …

`app/services/target_builder.py`

- `TargetUnitaryBuilder.build(name, qubits)` loops `steps`, reads `gate`, `order`, `theta`, calls `CirqGateMapper.apply` (same mapper as the trial path).

`app/utils/helpers.py`

- `build_target_truth_table` / related helpers use `TARGET_LIBRARY` when not building the circuit from scratch.

Where it "shows up" in the simulate flow

The client sends `target_unitary: "SWAP"` (a **string key**).
If `validate_target` is on, the backend builds the **target circuit** from `TARGET_LIBRARY["SWAP"].steps` — that is the level-definition shape.
The **trial** circuit never uses `steps[]`; it uses `gates` + `qubit_order` only.

What each DTO holds (and how it changes)

UnitaryDTO

`app/dto/unitary.py`

number_of_qubits

Positive integer — how many line qubits to allocate.

gates

List parallel to `qubit_order`. Each entry: a **string** (e.g. `"CNOT"`) or a **dict** `{"gate": "RX", "theta": 1.2}`. Nothing is "split out" later — the lists are passed through as-is.

qubit_order

`gates[i]` is applied using `qubit_order[i]` (wiring for that row).

Not included: `target_unitary` from JSON is a separate string; it is not a field on this DTO.

TruthTableDTO

`app/dto/truth_table.py`

Structure

`input: list[str]`, `output: list[str]` — parallel columns (same length when filled).

Created empty: `TruthTableDTO([], [])` for trial and for target at the start of `simulate_unitaries`.

Filled during simulation: each basis state run appends one row via `CircuitSimulator.simulate_and_update` (inputs like `|01⟩`, outputs from Cirq statevector notation).

Serialized for HTTP: `.to_dict()` → `{ "input": [...], "output": [...] }`.

Validation (no DTO)

`app/utils/unitary_payload.py`

validate_simulate_unitary_json

Checks the raw **dict**: required keys, list lengths, each gate string or `{"gate", "theta"?}`, numeric `theta`. Raises `ValueError` → API returns `ResponseBuilder.fail`. Does not construct a dataclass.

Response payload (dict, not a DTO class)

`controllers/simulate.py return value`

Contents

Message, `trial_truth_table` and `target_truth_table` as dicts from `TruthTableDTO.to_dict()`, plus `validation_mode`.

ASCII timeline (same path, file-oriented)

```
CLIENT
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  app/api/simulate.py  ·  Simulate.post()                             │
│    • unitary_info = request.get_json()          (plain dict)         │
└──────────────────────────────────────────────────────────────────────┘
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  validate_simulate_unitary_json(unitary_info)                        │
│    app/utils/unitary_payload.py                                      │
│    • Still dict — no DTO. Raises ValueError if shape is wrong.       │
└──────────────────────────────────────────────────────────────────────┘
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  UnitaryDTO(                                                         │
│      unitary_info["number_of_qubits"],                               │
│      unitary_info["gates"],        ← list[str | {"gate","theta"?}]   │
│      unitary_info["qubit_order"],                                    │
│  )                          app/dto/unitary.py                       │
│                                                                      │
│  target_name = unitary_info["target_unitary"]   ← str, not in UnitaryDTO │
└──────────────────────────────────────────────────────────────────────┘
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  simulate_unitaries(trial_dto, target_name, validate)                │
│    app/controllers/simulate.py                                       │
│      • TruthTableDTO([], []) ×2  (trial + target) — empty, then filled │
│      • FOR each basis state:                                         │
└──────────────────────────────────────────────────────────────────────┘
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  CircuitBuilder.construct_unitary_circuit(...)                       │
│    app/services/circuit_builder.py                                   │
│      • prepare_basis_state  → X gates for |1⟩ bits                   │
│      • build_circuit_base(trial_dto.gates, trial_dto.qubit_order, …) │
│            i=0: "H"     → theta None                                 │
│            i=1: dict    → gate "RX", theta 1.2                       │
│            i=2: "CNOT"  → theta None                                 │
└──────────────────────────────────────────────────────────────────────┘
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  CirqGateMapper.apply(gate_name, qubit_order[i], *qubits, theta=…)   │
│    app/config/gates.py                                               │
│      • RX + theta → cirq.rx(theta)(qubit)                            │
└──────────────────────────────────────────────────────────────────────┘
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  CircuitSimulator.simulate_and_update(...)                           │
│    app/services/simulator.py                                         │
│      • Appends one row to TruthTableDTO (mutate in place)            │
└──────────────────────────────────────────────────────────────────────┘
  │
  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Return { message, trial_truth_table, target_truth_table, … }        │
│    truth_table DTOs → .to_dict() for JSON                            │
└──────────────────────────────────────────────────────────────────────┘
```

Open this file in a browser (internet required once for the Mermaid CDN). If the diagram does not render, check that scripts are not blocked.